

O Jogo dos Macacos

¹Pontífica Universidade Católica do Rio Grande do Sul (PUCRS)
Porto Alegre – RS – Brazil
18 de Abril de 2023

Resumo. Este artigo descreve alternativas de solução para o primeiro problema proposto na disciplina de Algoritmos e Estruturas de Dados II cujo principal objetivo é criar um algoritmo capaz de simular as rodadas de uma versão rudimentar de bingo entre macacos retornando o resultado do vencedor. São propostas duas soluções para este problema. Desta forma, é analisado a eficiência de cada algoritmo, utilizando os estudos de casos e o tempo de execução de cada programa. Por último, é de suma importância ressaltar que, neste estudo, a refatoração de um código para o aprimoramento de sua eficiência é essencial devido a extensão dos arquivos analisados.

DETALHES DO JOGO?

1. Introdução

O problema a ser resolvido tem como finalidade a criação de um programa que simula uma brincadeira coletiva entre macaquinhos que trocam, entre si, cocos preenchidos com uma quantidade ímpar ou par de pedrinhas. Sob essa perspectiva, foram realizadas diversas técnicas que serão discutidas neste artigo para possíveis soluções.

NA IMPLEMENT.

Inicialmente fez-se a leitura de oito relatórios, descritos em formato .txt, que informavam as condições iniciais do jogo a fim de realizar esta simulação. Estes arquivos foram padronizados com estruturas contendo todas as informações necessárias para que o jogo ocorra sem erros. Desta forma, a leitura dos arquivos pode ser realizada de forma genérica, sem haver desorganização com as informações.

ISTO NÃO É MUITO. SIMPLIFICAR E ENCONTRAR!

```
Fazer 100000 rodadas
Macaco 0 par -> 4 impar -> 3 : 11 : 178 84 1 111 159 22 54 132 201 51 44
Macaco 1 par -> 0 impar -> 5 : 9 : 80 82 10 83 98 31 56 84 53
Macaco 2 par -> 3 impar -> 4 : 7 : 65 194 35 132 191 202 62
Macaco 3 par -> 0 impar -> 4 : 3 : 121 10 162
Macaco 4 par -> 0 impar -> 5 : 5 : 16 110 125 113 35
Macaco 5 par -> 2 impar -> 0 : 8 : 120 25 20 134 166 100 157 159
```

Figure 1. Exemplo da estrutura dos arquivos lidos.

A Figura 1 mostra a estrutura dos arquivos lidos durante a execução da simulação. Na primeira linha, temos a informação da quantidade de rodadas desejadas. Nas linhas sucessivas, temos a seguinte estrutura: A identificação do macaco, para quem ele enviará os seus cocos em caso de par ou ímpar, respectivamente, a quantidade de cocos do macaco no início do jogo e a quantidade de pedrinhas em cada um deles.

Durante as simulações, cada macaco dispõe da quantidade que desejar de cocos para serem preenchidos com quantas pedrinhas almejem. Os cocos serão distribuídos

entre os macaquinhos, conforme o número de pedrinhas pares ou ímpares, durante uma rodada, por quantas vezes forem necessárias. Sendo assim, um macaco pode distribuir seus cocos para mais de um macaquinho. 222

Ao final do número de rodadas estabelecidas no arquivo, o macaco proprietário da maior quantidade de cocos será considerado o vencedor da brincadeira. Assim, a simulação é encerrada e obtemos a solução para o problema anteriormente apontado.

2. Desenvolvimento

Neste relatório foram elaboradas duas soluções. O primeiro algoritmo criado cumpre os requisitos para a solução do problema, todavia o tempo de execução e a eficiência do algoritmo foram insatisfatórios. Em contrapartida, o segundo algoritmo demonstrou ser satisfatório em todos os requisitos.

2.1. Leitura dos Arquivos

A leitura dos arquivos permaneceu a mesma em ambas as soluções, visto que a divergência dos algoritmos está no armazenamento e na manipulação dos dados. Desta forma, esta seção abordará o processo de leitura dos dados de entrada de ambas as soluções encontradas.

Inicialmente, foi executada a leitura da primeira linha do arquivo¹, da qual foi retirada a informação da quantidade de rodadas e esta foi armazenada em uma variável. Em seguida, a leitura continuou sendo feita da seguinte forma:

- A cada iteração, uma linha é lida.
- A linha é dividida onde há espaços e armazenada em um vetor de Strings.
- Um objeto de tipo Macaco é criado e receberá o vetor de strings com as informações sobre o macaco da linha lida.
- Por último, este macaco é adicionado a uma lista de macacos que contém todos os macacos jogadores desta partida.

Assim, retiramos da linha exclusivamente as informações imprescindíveis, sendo elas: a identificação do macaco que receberá os cocos com pedrinhas pares e do que receberá os de pedrinhas ímpares e as informações sobre as pedrinhas, sejam elas pares ou ímpares. A partir deste ponto, a implementação dos algoritmos diverge e, portanto, elas serão explicadas separadamente.

2.2. Primeira Solução

Em primeira hipótese, entendeu-se que os macacos possuíam pilhas de cocos que seriam trocadas durante as partidas. Entretanto, ainda durante a compreensão do que deveria ser feito, notou-se que durante a implementação o uso da estrutura de pilhas não seria conveniente por conta da dificuldade de manusear elementos de dentro desse tipo estrutural.

Posteriormente, constatou-se que seria melhor pensar no armazenamento de coquinhos em formato de filas. Desta forma, decidiu-se que uma estrutura de lista seria mais adequada para a implementação do algoritmo. 6 Tenia Alunos, não parece. Faz mesmo tanta #?

Nesta primeira implementação, durante o armazenamento dos valores de entrada, as informações sobre os cocos e as pedrinhas eram introduzidas em estruturas do tipo

¹ Visualiza primeira linha da Figura 1 na seção 1

ArrayList de inteiros fazendo a separação do valor das pedrinhas entre pares e ímpares. Portanto, haviam duas listas de cocos para cada macaco: uma com pedrinhas de valores pares e outra de valores ímpares. ✓

Durante o processamento das jogadas, a cada iteração sobre um macaco da lista de jogadores eram criadas duas listas auxiliares que recebiam os valores das listas de pedrinhas pares e ímpares, respectivamente, deste macaco. Logo após, a lista auxiliar (par ou ímpar) era adicionada à lista (par ou ímpar) do macaco que deveria recebê-la. Por fim, as listas do macaco que estava enviando os seus cocos eram limpas.

O trecho de código da Figura 2, colocada abaixo, demonstra o exato trecho de código que ~~foi~~ explicado no parágrafo anterior. !!

Pensar em Pseudo-código

```
for (int j = 0; j < rodadas; j++) {  
    for (int i = 0; i < macacada.size(); i++) {  
        int impar = macacada.get(i).macacoImpar;  
        int par = macacada.get(i).macacoPar;  
  
        ArrayList<Integer> auxListaImpar = macacada.get(i).getlistaImpar();  
        ArrayList<Integer> auxListaPar = macacada.get(i).getlistaPar();  
  
        macacada.get(par).addPares(auxListaPar);  
        macacada.get(impar).addImpares(auxListaImpar);  
  
        macacada.get(i).cleanCocos();  
    }  
}
```

Figure 2. Trecho de código da iteração da lista de macacos

Em virtude deste trecho, a eficiência dessa solução não atendeu as expectativas iniciais. O uso de memória neste caso, se torna abundante levando em conta a extensão dos valores de cocos de cada macaco e da ~~quantidade~~ quantidade de vezes em que as listas são copiadas. *MAS FEZ TESTES? SEU RESULTADO? QUANTOS? TEMPOS?*

2.3. Segunda Solução

Dizem que estão lá no fim!
Devido à baixa eficiência na execução do programa da primeira solução, fez-se necessário uma nova visão sobre uma possível solução para o problema. Portanto, fazendo uma releitura do problema, percebeu-se que poderia haver uma nova abstração dos conceitos de cocos e pedrinhas no momento da realização do algoritmo.

Nesta segunda implementação, durante o armazenamento dos valores de entrada, verificava-se se o número de pedrinhas de um coco era par ou ímpar e incrementava-se uma variável do tipo int chamada de "cocosPares" ou "cocosImpares", conforme o resultado da verificação. Desta forma, não era mais necessário uma estrutura tão complexa quanto uma lista de array de inteiro, apenas o número de cocos pares e ímpares para chegar ao resultado esperado. *"ISTO É POSSÍVEL POR QUÊ..."*

Durante o processamento das jogadas, a cada iteração sobre um macaco da lista de jogadores eram criadas duas variáveis auxiliares do tipo int que recebiam os valores da quantidade de cocos pares e ímpares deste macaco. Logo após, a variável auxiliar (par ou ímpar) era adicionada à quantidade de cocos (pares ou ímpares) do macaco que deveria

Seu exemplo?

recebê-los. Por fim, os valores da quantidade de cocos do macaco que estava enviando os seus cocos eram limpadas. ☹️

```
for (int j = 0; j < rodadas; j++) {  
    for (int i = 0; i < macacada.size(); i++) {  
        int impar = macacada.get(i).impar;  
        int par = macacada.get(i).par;  
  
        int auxImpar = macacada.get(i).getCocosImpar();  
        int auxPar = macacada.get(i).getCocosPar();  
  
        macacada.get(par).addPares(auxPar);  
        macacada.get(impar).addImpares(auxImpar);  
  
        macacada.get(i).cleanCocos();  
    }  
}
```

Figure 3. Trecho de código da iteração da lista de macacos

Conforme visto neste trecho, a eficiência dessa solução atendeu as expectativas iniciais. Neste caso, como não estamos utilizando uma estrutura tão complexa conseguimos os resultados com um custo de execução inferior ao da solução anterior.

3. Resultados

Depois de implementar as duas soluções analisadas neste estudo, em Java e executá-las, obteve-se os seguintes resultados:

3.1. RESULTADOS DA PRIMEIRA SOLUÇÃO

Caso Teste	Total Cocos	Vencedor	Tempo (ms)
0050	2332	9	0,118
0100	15461	20	0,695
0200	74413	38	4,136
0400	145232	36	34,44
0600	230276	177	141
0800	182575	20	331,5
0900	433295	589	710,59
1000	581995	144	794,29

Observando a tabela acima é plausível constatar que conforme a quantidade de macacos foi sendo ampliada, o tempo de execução foi agravando. Essa consequência deve-se ao fato de que, nos arquivos em que havia uma quantidade maior de macacos, também existia uma quantidade maior de cocos a serem circulados e, portanto, a máquina demorava mais tempo para executar as cópias das listas.

3.2. RESULTADOS DA SEGUNDA SOLUÇÃO

Caso Teste	Total Cocos	Vencedor	Tempo (ms)
0050	2332	9	0,021
0100	15461	20	0,022
0200	74413	38	0,031
0400	145232	36	0,104
0600	230276	177	0,219
0800	182575	20	0,31
0900	433295	589	0,383
1000	581995	144	0,448

Conforme a tabela de resultados a cima, é plausível constatar que ainda que a quantidade de macacos foi sendo ampliada, o tempo de execução continuou sendo satisfatório. Por estar manuseando uma estrutura simples e que ocupa pouca memória em relação a uma lista sendo copiada diversas vezes, esta implementação alcançou de forma muito mais aprimorada a solução do problema.

3.3. Análise Comparativa

A fim de efetivamente assimilar o desvio entre os dois algoritmos propostos, foi efetuada uma breve análise sobre o desempenho de ambos. É imprescindível pontuar neste instante, que a extensão das medidas utilizadas na elaboração dos gráficos abaixo são diferentes devido a grande divergência entre o tempo de execução de cada programa.

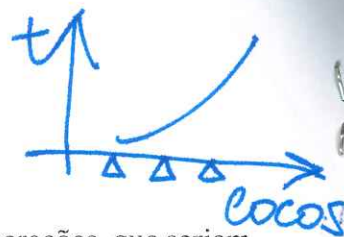


Figure 4. Gráfico tempo(s) x número de operações

De acordo com o gráfico da Figura 4, podemos perceber que, conforme o número de operações aumenta, o tempo de execução também tende a escalar. É possível percebermos esta subida pelo último caso que executa em quase 800 segundos, o que seria equivalente a 13 minutos. Ademais, é interessante observar que o número de cocos informado pelos arquivos também influencia na execução.

Simplificação

Ploton



Por exemplo, durante a execução dos dois últimos pontos de operações, que seriam os dois últimos casos de teste da tabela da subseção 3.1, a diferença no tempo de execução do algoritmo é diminuto se comparada a diferença do caso de 64M operações (ou caso teste de número 0800) para o penúltimo (caso teste de número 900).

Esta diferença deve-se por conta do número de cocos de cada caso teste. Como podemos observar na tabela da subseção 3.1, os dois últimos casos tem uma diferença entre o número de cocos excessivamente menor do que a diferença entre os casos de número 0800 a 0900.



Figure 5. Gráfico tempo(s) x número de operações

A respeito do segundo algoritmo, é possível observar um grande aprimoramento no tempo de execução. Diferente do gráfico da Figura 4, nesta Figura 5 o tempo de execução do último caso não chega a 0,5 segundos. Portanto, podemos concluir que, enquanto o último caso estiver sendo executado pelo programa da primeira solução, este mesmo caso pode rodar praticamente 1600 vezes no segundo algoritmo até que o resultado do macaco vencedor seja estipulado pelo primeiro programa.

4. Conclusões

Os resultados obtidos com as implementações e análises propostas por este estudo foram satisfatórias, tendo em vista que o problema proposto foi solucionado em tempo razoável e apresentando dados corretos.

Não obstante, é importante destacar que há possibilidades de melhoria na eficiência dos algoritmos propostos por este estudo. Uma possibilidade de aprimoramento seria a tentativa de reconhecimento de um padrão dentro das jogadas dos macacos. Desta forma, uma fórmula para essa sequência de partidas, possivelmente, poderia ser criada e implementada no algoritmo gerando uma eficiência maior na execução.

Apresentação				
Texto (peso 2)	Apresentação clara e apropriada para um relatório científico. O leitor segue com facilidade a linha de raciocínio. Os tópicos são ligados logicamente. Escrita sem erros de gramática, pontuação, ortografia. Texto fluído sem dificuldade nem excesso. Valor 2	Apresentação clara em geral. Texto é quase sempre conexo. Existem erros de estilo e gramática. Alguns pontos são confusos ou desconexos. Valor 1.4	O leitor segue o texto com dificuldade. Organização é deficiente e texto parece desconexo. Tópicos não são claros, sentenças são confusas e desorganizadas. Erros em estilo e gramática acontecem o bastante para distrair o leitor. Valor 0.6	Apresentação confusa e obscura. Escrita é inapropriada e com erros de linguagem frequentes que perturbam a leitura a ponto de ser difícil extrair o significado. Não há conexão entre ideias e/ou sentenças. O leitor não consegue seguir o texto nem extrair os principais pontos apresentados. Valor 0
Figuras e Tabelas (peso 1)	Apóiam o texto de forma efetiva. Informação é apresentada com clareza nas figuras e nas legendas, gráficos tem unidades. São usadas para explicar o algoritmo. Valor 1	Fornecem boas informações adicionais, mas podem ter pouca clareza ou falhas nas legendas e unidades. Não explicam bem o algoritmo usado. Valor 0.7	Difíceis de entender e/ou pouco informativas. Não são usadas para explicar o algoritmo. Unidades em gráficos são faltantes. O texto deve ser consultado para sua compreensão. Valor 0.3	Figuras e tabelas não são usadas quando deveriam ser, ou são mal apresentadas a ponto de atrapalhar o leitor. Legendas ausentes. Não ilustram os pontos apresentados no texto. Valor 0
Algoritmos				
Desenvolvimento (peso 2)	Apresenta informações do contexto. Explica a importância e interesse do assunto. O problema é bem definido, a solução é obtida passo a passo a partir do problema e resultados e conclusões estão bem apresentados. Informação é precisa e clara. Valor 2	Fornecer informações gerais sobre o assunto, mas alguma informação relevante está ausente ou o significado não é explicado com clareza. Solução não é bem vinculada ao problema. Descrição dos resultados é geralmente clara, podendo haver pontos confusos. Não há erros significativos. Valor 1.4	Informações sobre o contexto, relevância e/ou significado insuficientes. Problema não é definido com clareza e faltam detalhes. Solução não está bem vinculada ao problema, sua obtenção é pouco clara. Alguma informação é acurada, mas existem erros suficientes para distrair o leitor. Valor 0.6	Praticamente não fornece informações sobre o contexto, relevância e/ou significado. Informação é imprecisa, não é fundamentada ou contém muitos erros. Discussão é difícil de seguir. O leitor não aprende na discussão. Valor 0
Algoritmos (peso 3)	São apresentados com clareza, fundamentados e explicados com detalhes. O pseudo-código é claro e tão simples quanto possível. Exemplos são fornecidos e seu funcionamento é explicado com clareza. Exceções são tratadas. Valor 3	Existem inconsistências na descrição do algoritmo, mas elas não atrapalham o entendimento. Pseudo-código poderia ser melhorado ou reduzido. Exemplos são muito simples ou não são claros. Valor 2.1	Inconsistências na descrição dificultam seriamente a compreensão do algoritmo. O pseudo-código é obscuro, insuficiente ou confuso. Exemplos são difíceis de seguir e não ajudam no entendimento. Valor 0.9	Algoritmo tem erros de concepção. Pseudo-código possivelmente com erros, excessivo, ausente ou em blocos de linguagem de programação. Exemplos ou resultados contêm erros ou não são apresentados. Algoritmo está incorreto. Valor 0
Eficiência (peso 2)	Análise de eficiência é obtida com detalhes ou o algoritmo pode ser considerado eficiente. Valor 2	Análise de eficiência tem falhas na apresentação ou algoritmo poderia ser tornado mais eficiente com relativa facilidade. Valor 1.4	Análise de eficiência tem erros sérios ou o algoritmo é ineficiente a ponto de comprometer seu uso além dos casos mais simples. Valor 0.6	Análise de eficiência inexistente ou o algoritmo não pode ser usado na prática. Valor 0
Atenção: nem todos os problemas assinalados podem estar presentes no seu trabalho, mas eles servem para dar uma ideia do tipo de problema que está nesta categoria.				

7,7